基于ABM-MARL的酒店-OTA动态定价博弈系统设计文档

> **算法基础**: Cross-Entropy Method (CEM)
>
> **系统架构**: 基于智能体建模（ABM）的多智能体强化学习（MARL）博弈框架
>
> **核心场景**: 酒店双渠道定价与OTA补贴策略的Stackelberg博弈

目录

1. 系统概述
2. 项目架构与模块组织
3. ABM环境建模
4. 酒店Agent设计
5. OTA Agent设计
6. Agent与环境的交互机制
7. 奖励函数设计
8. 伪代码
9. 数据来源与预处理
10. 实验配置与超参数
11. Cross-Entropy Method算法实现
12. 关键设计决策总结

---

1. 系统概述

1.1 研究背景

在现代酒店行业中，酒店通常同时通过线上（OTA渠道，如携程、美团等）和线下（前台直销）两种渠道进行客房销售。酒店需要在两个渠道上分别制定价格策略，而OTA平台则通过佣金收入和补贴策略来影响线上渠道的最终售价，以此吸引更多客户通过OTA渠道预订。

这构成了一个典型的**双层博弈问题**：酒店（领导者）制定线上基础价格和线下价格，OTA（跟随者）基于酒店的定价决定补贴比例。双方的策略相互影响，最终通过消费者的预订行为体现为各自的收益。

1.2 系统目标

本系统旨在构建一个完整的仿真与学习框架，实现以下目标：

1. **真实市场模拟**：通过ABM（Agent-Based Model）方法，基于历史酒店预订数据模拟消费者的到达、决策和预订行为，还原真实的市场需求响应。

2. **多智能体博弈学习**：使用MARL（Multi-Agent Reinforcement Learning）框架，让酒店Agent和OTA Agent在模拟环境中进行策略博弈，各自学习最优的定价/补贴策略。
3. **策略均衡分析**：通过大量训练迭代，观察双方策略的收敛情况，分析博弈均衡下的价格水平、渠道分配和利润分配。

1.3 技术架构总览

系统采用三层架构设计：

```
┌──────────────────────────────────────────────────────────────┐
│                          训练控制层                            │
│  GameTrainer: 管理训练流程、策略更新、数据记录                  │
├──────────────────────────────────────────────────────────────┤
│                          决策层（MARL）                        │
│  ┌──────────────────┐    ┌──────────────────┐                │
│  │ HotelAgent       │    │  OTAAgent        │                │
│  │ (双CEM算法)      │    │  (单CEM算法)     │                │
│  │ 输出:线上基础价格 │    │  输出:补贴比例   │                │
│  │      线下价格    │    │  (0%-80%)        │                │
│  └────────┬─────────┘    └────────┬─────────┘                │
│           │          策略交互      │                          │
│           └──────────┬────────────┘                          │
├──────────────────────┼──────────────────────────────────────┤
│                      ▼                                       │
│                    环境层（ABM）                               │
│  ┌──────────────────────────────────────────────┐            │
│  │ HotelEnvironment                             │            │
│  │ ├── 状态管理（库存、季节、日期类型）          │            │
│  │ ├── 价格窗口管理（91天滚动窗口）              │            │
│  │ └── HotelABMModel（Mesa框架）                 │            │
│  │       ├── 消费者生成（泊松过程）              │            │
│  │       ├── 消费者决策（效用函数）              │            │
│  │       └── 库存约束与预订处理                  │            │
│  └──────────────────────────────────────────────┘            │
└──────────────────────────────────────────────────────────────┘
```

---

2. 项目架构与模块组织

2.1 目录结构

```
ABM_MARL_hotel_pricing/
├── configs/
|   └── config.py                        # 全局配置中心
├── src/
|   ├── algorithms/                      # RL算法实现
|   |   ├── base_algorithm.py            # 算法抽象基类
|   |   ├── cem.py                       # 表格版CEM算法（本文档核心）
|   |   └── cem_nn.py                    # 神经网络版CEM算法
|   ├── agent/                           # 智能体实现
|   |   ├── hotel_agent_dual_channel.py  # 双渠道酒店Agent
|   |   └── ota_agent.py                 # OTA Agent
|   ├── environment/                     # 环境模拟
|   |   ├── hotel_env.py                 # 酒店RL环境
|   |   └── abm_customer_model.py        # ABM消费者行为模型
|   ├── training/
|   |   └── game_trainer.py              # 博弈训练器
|   └── utils/
|       └── training_monitor.py          # 训练监控与可视化
├── experiments/
|   └── train_game.py                    # 训练入口脚本
├── datasets/
|   └── hotel_bookings.csv               # 历史预订数据
└── outputs/
    ├── models/                          # 训练后的模型参数（JSON）
    ├── results/                         # 训练统计数据（CSV）
    ├── figures/                         # 训练结果可视化图表
    └── tensorboard_logs/                # TensorBoard训练日志
```

2.2 核心模块依赖关系

```
train_game.py（入口）
  └── game_trainer.py（训练流程编排）
        ├── hotel_env.py（RL环境封装）
        |     └── abm_customer_model.py（消费者行为模拟）
        ├── hotel_agent_dual_channel.py（酒店决策Agent）
        |     └── cem.py / cem_nn.py（CEM算法实现）
        └── ota_agent.py（OTA决策Agent）
              └── cem.py / cem_nn.py（CEM算法实现）
```

---

3. ABM环境建模

ABM（Agent-Based Model）是本系统的核心环境组件，基于Mesa框架实现，负责模拟消费者的到达、特征生成、预订决策等行为，为强化学习Agent提供真实的市场反馈信号。

3.1 消费者生成机制

3.1.1 每日消费者到达（泊松过程）

每日潜在消费者数量服从泊松分布，到达率 λ_m 按月份从历史数据中估计：

$$N_{\text{day}} \sim \text{Poisson}(\lambda_m)$$

其中 m 为当前月份（1-12）。月份由仿真天数简化计算：$m = (\lfloor \text{day}/30 \rfloor \mod 12) + 1$。

到达率 λ_m 的计算方法：基于 `hotel_bookings.csv` 中各月份的预订记录总数除以30（简化的月天数），得到日均到达率。

```python
# 具体实现（config.py）
monthly_rates[month] = monthly_counts[month] / 30.0
```

3.1.2 消费者特征生成

每个消费者 i 被赋予如下特征向量 $(\text{lead_time}_i, T_{\text{stay},i}, \text{WTP}_i, \beta_i, \text{type}_i)$：

1) 提前预订期（Lead Time） L_i

提前预订期从历史数据拟合的经验分布中采样。系统首先从 `hotel_bookings.csv` 中统计各提前天数（0-90天）对应的预订频次，构建离散概率分布：

$$L_i \sim \text{Empirical}(p_0, p_1, \ldots, p_{90})$$

其中 $p_d = \frac{\text{count}(L=d)}{\sum_{d'=0}^{90} \text{count}(L=d')}$。

采样后对结果进行裁剪，确保 $L_i \in [0, \text{booking_window_days} - 1]$，即最多提前90天预订。

2) 目标入住日期 $T_{\text{stay},i}$

$$T_{\text{stay},i} = \text{current_day} + L_i$$

消费者的目标入住日期等于当前仿真日期加上其提前预订期。

3) 最高支付意愿（Willingness To Pay, WTP） WTP_i

支付意愿服从正态分布，参数由 `hotel_bookings.csv` 中未取消订单的平均日房价（ADR）拟合：

$$\text{WTP}_i \sim \mathcal{N}(\mu_{\text{adr}}, \sigma_{\text{adr}})$$

拟合时过滤掉已取消订单和异常值（$\text{ADR} \leq 0$ 或 $\text{ADR} \geq 500$），并对采样结果设置下限 $\text{WTP}_i \geq 10.0$。

4) 消费者类型 type_i

$$\text{type}_i \sim \text{Categorical}(\text{ota}: 0.3, \text{ota+direct}: 0.7)$$

即30%的消费者仅通过OTA线上渠道预订（`ota`类型），70%的消费者可同时看到线上和线下价格并选择效用更高的渠道（`ota+direct`类型）。

> **关键设计说明**：`ota` 类型消费者只会比较线上渠道价格；`ota+direct` 类型消费者会同时评估线上和线下两个渠道的效用，选择效用更高的渠道进行预订。

3.2 消费者预订决策模型

消费者的预订决策基于**效用函数**，核心思想是：消费者评估预订行为的"值得程度"，当效用超过阈值时执行预订。

3.2.1 效用函数

消费者 i 面对价格 P 时的预订效用为：

$$U_i(P) = (\text{WTP}_i \cdot \delta - P) + \frac{\gamma}{L_i + 1}$$

其中各项含义如下：

符号	含义	说明
WTP_i	最高支付意愿	消费者愿意为一晚住宿支付的最高价格
δ	渠道折扣系数	线上渠道 $\delta = 0.8$（消费者认为线上价值打8折），线下渠道 $\delta = 1.0$
P	渠道报价	线上为OTA最终价格，线下为酒店直销价格
γ	紧迫性权重	默认值20，控制提前期对预订决策的影响强度
L_i	提前预订天数	距离入住日还有多少天

效用函数的经济学解读：

- **第一项**$(\text{WTP}_i \cdot \delta - P)$：经济盈余，反映消费者从预订中获得的净"赚到"感。当WTP远高于价格时，消费者倾向预订。
- **第二项**$\frac{\gamma}{L_i+1}$：紧迫性效用，提前期越短，入住需求越紧迫，预订意愿越强。

3.2.2 渠道选择逻辑

消费者的渠道选择取决于其类型：

- `ota` **型消费者**：仅计算线上效用 $U_{\text{online}} = (\text{WTP}_i \times 0.8 - P_{\text{online}}) + \frac{\gamma}{L_i+1}$
- `ota+direct` **型消费者**：同时计算两个渠道的效用，选择效用更高的渠道：

- $U_{\text{online}} = (\text{WTP}_i \times 0.8 - P_{\text{online}}) + \frac{\gamma}{L_i+1}$
- $U_{\text{offline}} = (\text{WTP}_i \times 1.0 - P_{\text{offline}}) + \frac{\gamma}{L_i+1}$
- 最终选择 $\max(U_{\text{online}}, U_{\text{offline}})$ 对应的渠道

3.2.3 预订决策规则

$$\text{Book} = \begin{cases} \text{True}, & \text{if } U^* > \theta \\ \text{False}, & \text{otherwise} \end{cases}$$

其中 U^* 为选定渠道的效用值，$\theta = -15$ 为预订阈值。阈值为负数意味着即使消费者觉得价格略高于自己的心理价位，在紧迫性足够高时仍可能做出预订。

3.3 库存约束

即使消费者决定预订，预订能否成功还取决于目标入住日期的剩余库存：

$$\text{Booking Success} = \begin{cases} \text{True}, & \text{if available_rooms}[T_{\text{stay},i}] > 0 \\ \text{False}, & \text{otherwise} \end{cases}$$

成功预订后，对应日期的可用库存减1。系统维护一个以入住日期为键的库存字典 `daily_available_rooms`，确保每个入住日期的库存约束独立管理。

3.4 预订记录

每笔成功的预订被记录为 `BookingRecord`，包含以下信息：

字段	类型	说明
`customer_id`	int	消费者唯一ID
`booking_date`	int	预订发生的仿真日期
`target_date`	int	目标入住日期
`paid_price`	float	成交价格
`wtp`	float	消费者WTP
`customer_type`	str	最终选择的渠道（`online`/`offline`）

3.5 每日模拟流程

ABM模型的 `simulate_day()` 方法执行一天的完整模拟：

```
输入：线上价格窗口，线下价格窗口，各日库存
输出：当日预订统计（按渠道、按提前天数分组）

1. 调用 generate_daily_customers() 生成当日潜在消费者
2. 对每个消费者：
   a. 计算其目标入住日期对应的提前天数 days_ahead
   b. 从价格窗口中获取该提前天数对应的价格
   c. 调用 make_booking_decision() 进行决策
   d. 若决定预订且库存充足，则记录预订并扣减库存
3. 汇总当日统计：
   - 线上/线下新增预订量
   - 按 day_offset 分组的预订量与收入
   - 当日总收入
```

---

4. 酒店Agent设计

4.1 设计理念

酒店Agent（`HotelAgentDualChannel`）是一个**双渠道定价决策器**，使用两个独立的CEM算法分别为线上和线下渠道制定价格。

设计核心考量：

- **渠道独立性**：线上与线下渠道面向不同消费群体，价格策略应独立优化
- **佣金意识**：线上基础价格需覆盖OTA佣金成本，否则酒店实际收入低于线下
- **博弈响应**：需考虑OTA的补贴行为对最终线上价格的影响

4.2 状态空间设计

系统采用离散状态空间，由三个维度组合构成：

$$\mathcal{S} = \{\text{inventory_level}\} \times \{\text{season}\} \times \{\text{weekday}\}$$

4.2.1 库存水平（Inventory Level）

当日入住日（窗口第0天）的剩余库存被离散化为3个等级：

等级	范围	语义
0（低库存）	$\text{rooms} \leq \lfloor C/3 \rfloor$	客房紧张，应提价
1（中等库存）	$\lfloor C/3 \rfloor < \text{rooms} \leq \lfloor 2C/3 \rfloor$	库存适中
2（高库存）	$\text{rooms} > \lfloor 2C/3 \rfloor$	客房充裕，可降价促销

其中 C 为酒店总客房数（默认200间）。

4.2.2 季节（Season）

基于当前仿真天数推算月份，划分为三个季节：

等级	月份	语义
0（淡季）	11月、12月、1月、2月	需求低迷
1（平季）	3月-5月、9月-10月	需求中等
2（旺季）	6月、7月、8月	需求旺盛

月份计算公式：$m = (\lfloor \text{day}/30 \rfloor \mod 12) + 1$

4.2.3 日期类型（Weekday/Weekend）

等级	条件	语义
0（工作日）	$\text{day} \mod 7 \notin \{5, 6\}$	周一至周四
1（周末）	$\text{day} \mod 7 \in \{5, 6\}$	周五至周日

4.2.4 状态编码

三个维度组合为标量索引：

$$\text{state_idx} = \text{inventory_level} \times 6 + \text{season} \times 2 + \text{weekday}$$

基础状态空间大小：$3 \times 3 \times 2 = 18$ 种状态。

4.2.5 决策阶段扩展

在博弈训练中，91天的预订窗口被划分为多个**决策桶**（Decision Buckets），每个桶对应一个决策阶段。例如默认配置 `"0|1|2-3|4-6|7-13|14-29|30-59|60-90"` 将窗口划分为8个阶段。

扩展后的状态索引：

$$\text{state_idx_extended} = \text{base_state_idx} \times K + \text{stage_id}$$

其中 K 为决策阶段数量。**酒店Agent总状态空间**：$18 \times K$ 种状态。

4.3 动作空间设计

酒店Agent采用连续动作空间，输出两个独立的价格值：

动作维度	范围	说明
$P_{\text{online_base}}$	[80, 180] 元	给OTA的线上基础价格
P_{offline}	[80, 180] 元	线下直销价格

两个价格分别由两个独立的CEM算法决策。

4.4 收益计算

$$R_{\text{hotel}} = \underbrace{B_{\text{online}} \times P_{\text{online_base}} \times (1-c)}_{\text{线上收益（扣佣金后）}} + \underbrace{B_{\text{offline}} \times P_{\text{offline}}}_{\text{线下收益}}$$

其中：

- B_{online}：线上渠道预订量
- B_{offline}：线下渠道预订量
- c：佣金率（默认 20%）

---

5. OTA Agent设计

5.1 设计理念

OTA Agent代表在线旅行社平台，其核心决策变量是**补贴比例**——即从佣金收入中拿出多少比例用于降低线上售价以吸引消费者。

OTA面临的权衡：

- **补贴越高** → 线上价格越低 → 吸引更多消费者 → 预订量上升 → 但单笔利润下降
- **补贴越低** → 保留更多佣金 → 单笔利润高 → 但线上价格竞争力下降 → 预订量可能减少

5.2 OTA状态空间

OTA的状态空间比酒店更大，包含5个维度：

$$\mathcal{S}_{\text{OTA}} = \{\text{price_gap}\} \times \{\text{inventory}\} \times \{\text{season}\} \times \{\text{weekday}\}$$

价格差异等级（$P_{\text{online_base}} - P_{\text{offline}}$）：

等级	条件	含义
0	gap < 0	线上基础价格低于线下（异常）
1	0 ≤ gap < 5	价差极小
2	5 ≤ gap < 15	价差较小
3	15 ≤ gap < 25	价差中等
4	gap ≥ 25	价差较大

状态编码公式：

$$\text{state_idx} = \text{price_gap_level} \times 18 + \text{inventory_level} \times 6 + \text{season} \times 2 + \text{weekday}$$

OTA基础状态空间大小：$5 \times 3 \times 3 \times 2 = 90$ 种状态，扩展后为 $90 \times K$。

5.3 OTA动作空间

OTA Agent输出一个连续的补贴比例：

动作维度	范围	说明
r_{subsidy}	[0, 0.8]	补贴占佣金收入的比例

5.4 价格计算链

酒店和OTA的动作共同决定最终的线上售价：

$$P_{\text{online_final}} = P_{\text{online_base}} - P_{\text{online_base}} \times \text{commission_rate} \times r_{\text{subsidy}}$$

即：最终线上价格 = 酒店基础价格 - OTA从佣金中拿出的补贴金额。

例如：酒店设定线上基础价格120元，佣金率30%，OTA补贴比例50%：

- 佣金收入 = 120 × 30% = 36元
- 补贴金额 = 36 × 50% = 18元
- 最终线上价格 = 120 - 18 = 102元

5.5 利润计算

$$\Pi_{\text{OTA}} = \underbrace{B_{\text{online}} \times P_{\text{online_base}} \times c}_{\text{佣金收入}} \times (1 - r_{\text{subsidy}})$$

等价于：

$$\Pi_{\text{OTA}} = \text{佣金收入} - \text{补贴支出}$$

其中 $\text{补贴支出} = \text{佣金收入} \times r_{\text{subsidy}}$。

---

6. Agent与环境的交互机制

6.1 交互总体流程

每天的交互涉及多个组件的协作：

```
┌──────────────┐        ①获取状态        ┌──────────────┐
| Environment  |◄──────────────────────  |  GameTrainer  |
| (hotel_env)  |                         |  (训练器)      |
|   ├─ state   |  ②状态传递             |               |
|   ├─ inventory|──────────────────────► | ┌──────────┐  |
|   └─ ABM     |                         | |HotelAgent | |
|              |                         | | → prices  | |
|              |  ③价格传递             | └──────────┘  |
|              |◄──────────────────────|               |
|              |                         | ┌──────────┐  |
|              |  ④补贴传递             | |OTA Agent  | |
|              |◄──────────────────────| | → subsidy | |
|              |                         | └──────────┘  |
|              |                         |               |
|   ABM模拟    |  ⑤模拟结果            |               |
|   消费者到达 |──────────────────────► |  ⑥计算奖励    |
|   预订决策   |                         |  ⑦更新Agent   |
|   库存扣减   |                         |               |
└──────────────┘                         └──────────────┘
```

6.2 91天滚动预订窗口

系统维护一个91天的滚动窗口（`booking_window_days = 91`），对应提前预订期0-90天：

```
Day t:    [t, t+1, t+2, ..., t+90]    ← 当前窗口
Day t+1:  [t+1, t+2, t+3, ..., t+91]  ← 窗口滚动
```

每天结束时：

1. 移除窗口最左端（当天已过期）
2. 在最右端添加新的一天（初始库存为满房）
3. 同步滚动价格窗口

每个窗口位置独立维护库存和价格，ABM消费者根据自身的提前预订期查找对应位置的价格。

6.3 决策桶（Decision Buckets）机制

91天的预订窗口被划分为多个决策桶，每个桶内的所有日期偏移使用相同的定价策略。

默认分桶配置：`"0|1|2-3|4-6|7-13|14-29|30-59|60-90"`

桶编号	偏移范围	天数	语义
0	0	1天	当天入住
1	1	1天	明天入住
2	2-3	2天	后天-大后天

桶编号	偏移范围	天数	语义
3	4-6	3天	近一周
4	7-13	7天	一至两周
5	14-29	16天	两周至一个月
6	30-59	30天	一至两个月
7	60-90	31天	两至三个月

设计意图：越近期的日期，定价决策越细致（单日级别）；越远期的日期，由于不确定性更大，使用更粗粒度的决策区间。

6.4 滚动窗口与状态同步

每天结束后，环境执行窗口滚动：

```
窗口滚动前: [Day_t, Day_{t+1}, ..., Day_{t+90}]
窗口滚动后: [Day_{t+1}, Day_{t+2}, ..., Day_{t+91}]
```

同步操作包括：

1. 从ABM模型回同步最新库存（ABM中已实时扣减）
2. 移除窗口最左端（今天已过期）
3. 在最右端添加新日期（初始库存=满房）
4. 价格窗口同步滚动
5. 决策桶的累计预订量同步滚动

6.5 详细交互时序

以一天（Day t）的交互为例：

Step 1: 决策制定阶段（按桶分组）

对每个决策桶 (s_k, e_k)，$k = 0, 1, \ldots, K-1$：

1. 获取该桶代表日期偏移的环境状态：

   ```python
   state = env._get_state_for_day_offset(ref_offset)
   state['stage_id'] = k  # 附加阶段标识
   ```

2. 酒店Agent选择价格：

   ```python
   price_online_base, price_offline = hotel_agent.select_action(state)
   ```

3. OTA Agent选择补贴：

```python
subsidy_ratio = ota_agent.select_action(price_online_base, price_offline, state)
```

4. 计算最终线上价格：

```python
price_online_final = price_online_base - price_online_base * commission_rate * subsidy_ratio
```

5. 将价格填充到桶内的所有日期偏移位置

Step 2: 环境模拟阶段

将完整的91天价格窗口传入环境：

```python
actions_window = [[price_online_final[i], price_offline[i]] for i in range(91)]
next_state, revenue, done, info = env.step(actions_window)
```

环境内部：

1. 将价格窗口同步到ABM模型
2. 将库存状态同步到ABM模型
3. ABM执行当日模拟（消费者生成→决策→预订）
4. 返回按 `day_offset` 分组的预订统计

Step 3: 奖励计算与Agent更新

1. 从 `info['bookings_by_day_offset']` 中提取各偏移位置的预订量
2. 按偏移位置累计预订量
3. 在决策桶的触发日到达时：
   - 计算该桶累计的酒店收益和OTA利润
   - 构建奖励信号
   - 更新对应Agent的CEM参数

---

7. 奖励函数设计

7.1 基本收益计算

酒店原始收益：

$$R_{\text{hotel}}^{\text{raw}} = B_{\text{online}} \times P_{\text{online_base}} \times (1 - c) + B_{\text{offline}} \times P_{\text{offline}}$$

OTA原始利润：

$$\Pi_{\text{OTA}}^{\text{raw}} = B_{\text{online}} \times P_{\text{online_base}} \times c \times (1 - r_{\text{subsidy}})$$

系统总利润：

$$\Pi_{\text{system}} = R_{\text{hotel}}^{\text{raw}} + \Pi_{\text{OTA}}^{\text{raw}}$$

7.2 混合奖励机制

为了引导Agent在自利和合作之间取得平衡，系统采用加权混合奖励：

酒店Agent的奖励：

$$\text{Reward}_{\text{hotel}} = \alpha \times R_{\text{hotel}}^{\text{raw}} + (1 - \alpha) \times \Pi_{\text{system}}$$

OTA Agent的奖励：

$$\text{Reward}_{\text{OTA}} = \alpha' \times \Pi_{\text{OTA}}^{\text{raw}} + (1 - \alpha') \times \Pi_{\text{system}}$$

其中：

- α（`reward_hotel_ratio`）：酒店个体收益权重，默认值0
- α'（`reward_ota_ratio`）：OTA个体收益权重，默认值1

默认配置的经济含义：

α 值	含义
$\alpha = 1$	完全自私，Agent只关心自己的收益
$\alpha = 0$	完全利他/系统导向，Agent优化系统总利润
$0 < \alpha < 1$	在个体利益和系统利益之间权衡

当前默认配置（$\alpha = 0, \alpha' = 1$）意味着：

- 酒店Agent的奖励 = 系统总利润（酒店被引导向系统最优方向定价）
- OTA Agent的奖励 = OTA自身利润（OTA以自身利润最大化为目标）

7.3 奖励的时间聚合

奖励不是逐天给予Agent的，而是按**决策桶的触发时刻**聚合：

```
桶 k 的触发日到达时：
  reward_k = 该桶内所有日期偏移的累计预订产生的总奖励
```

这意味着Agent看到的是一段时间内（桶的跨度）策略执行的累计效果，有助于减少噪声和稳定学习。

---

8. 伪代码

```
输入: 历史数据, 训练轮数 E, 预订窗口 W=91, 决策桶配置
输出: 训练后的酒店Agent和OTA Agent

1. 初始化环境 env, 酒店Agent, OTA Agent
2. 解析决策桶 → K 个阶段

3. FOR episode = 1 to E:
   a. env.reset()
   b. 对每个桶 k ∈ {0,...,K-1}:
      - 获取桶 k 代表位置的状态 s_k
      - 酒店选择价格: (P_on[k], P_off[k]) = hotel.select_action(s_k)
      - OTA选择补贴: r_sub[k] = ota.select_action(P_on[k], P_off[k], s_k)
      - 初始化桶 k 的累计预订量 = 0

   c. FOR day = 0 to 364:
      (i)  对每个触发偏移 off:
           - 若该偏移的累计预订 > 0:
             · 计算酒店收益和OTA利润
             · 构建奖励信号
             · 更新酒店CEM和OTA CEM
             · 重置累计预订 = 0
           - 重新决策该偏移的价格和补贴

      (ii) 计算最终线上价格窗口
      (iii) 传入环境: env.step(actions_window)
      (iv) 从返回的 bookings_by_day_offset 中累计各偏移的预订量

      (v)  按 update_frequency 定期调用 end_episode()
           → 触发CEM分布更新和标准差衰减

      (vi) 滚动所有窗口数组

   d. 处理剩余未更新的桶（episode结束时的最终更新）
   e. 调用 end_episode() 完成episode级参数更新
   f. 记录episode统计信息

4. 保存模型参数
```

---

9. 数据来源与预处理

9.1 数据集

系统使用 **Hotel Booking Demand** 数据集（`hotel_bookings.csv`），并仅使用其中 **City Hotel**（城市酒店）的子集。

9.2 从历史数据中提取的参数

提取内容	使用的字段	处理方法
月度到达率 λ_m	`arrival_date_month`	按月分组计数后除以30
提前期经验分布	`lead_time`	统计0-90天各天数的频率，构建离散概率分布
WTP分布参数	`adr`	未取消订单的ADR的均值和标准差

9.3 数据过滤

- 仅使用 `hotel == 'City Hotel'` 的记录
- WTP拟合时排除已取消订单（`is_canceled == 1`）
- 排除ADR异常值（≤ 0 或 ≥ 500）
- 提前期裁剪到 [0, 90] 天

---

10. 实验配置与超参数

10.1 环境参数

参数	值	说明
`initial_inventory`	200	酒店总客房数
`booking_window_days`	91	预订窗口长度（含当天）
`episode_days`	365	每个episode模拟天数
`cost_per_room`	20	每间客房成本

10.2 博弈系统参数

参数	值	说明
`commission_rate`	0.20	OTA佣金率
`subsidy_ratio_min`	0.0	OTA最低补贴比例
`subsidy_ratio_max`	0.8	OTA最高补贴比例
`online_price_range`	[80, 180]	线上基础价格范围
`offline_price_range`	[80, 180]	线下价格范围
`training_mode`	simultaneous	默认同步训练

10.3 ABM消费者参数

参数	值	说明
`urgency_weight` γ	20	紧迫性权重
`booking_threshold` θ	-15	预订效用阈值
`customer_type_ratio`	(0.3, 0.7)	线上/线下消费者比例
`online_discount_ratio` δ	0.8	线上渠道WTP折扣系数
`noise_std`	12.0	效用噪声标准差

10.4 CEM算法参数

参数	酒店Agent	OTA Agent
`n_samples`	100	100
`elite_frac`	0.3	0.3
`initial_std`	20.0	0.2
`min_std`	2.0	0.02
`std_decay`	0.99	0.99
`memory_size`	100	100
`discount_factor`	0.99	0.99

10.5 训练参数

参数	默认值	说明
`episodes`	200	训练总轮数
`update_frequency`	30	CEM分布更新频率（每N天）
`decision_buckets`	`0|1|2-3|4-6|7-13|14-29|30-59|60-90`	提前期分桶配置
`reward_hotel_ratio`	0.0	酒店个体收益权重
`reward_ota_ratio`	1.0	OTA个体收益权重

---

11. Cross-Entropy Method算法实现

11.1 算法概述

Cross-Entropy Method（CEM）是一种基于采样的黑箱优化方法，不需要计算梯度，通过迭代地采样-筛选-更新来搜索最优策略。其核心思想是：

1. 维护每个状态的动作分布（高斯分布）
2. 从分布中采样多个候选动作
3. 根据收益选出"精英"样本
4. 用精英样本的统计量更新分布参数

CEM特别适合本系统的原因：

- **无需梯度**：环境奖励通过ABM模拟获得，不可微分
- **高稳定性**：不存在Q值高估等问题，训练过程稳定
- **适合随机环境**：ABM环境具有固有随机性（泊松到达、随机WTP等），CEM天然适应

11.2 数学形式化

对每个状态 s，维护一个高斯分布参数 (μ_s, σ_s)：

$$a \sim \mathcal{N}(\mu_s, \sigma_s^2)$$

11.2.1 经验收集

每次在状态 s 下执行动作 a 并获得奖励 r 后，将经验三元组 (s, a, r) 存储到状态 s 对应的经验回放缓冲区（FIFO队列，容量100）。

11.2.2 分布更新

在每个episode结束时（或按固定频率），对所有被访问过的状态执行分布更新：

1. **取最近 N 个经验**：$\{(a_1, r_1), (a_2, r_2), \ldots, (a_N, r_N)\}$，其中 $N = \min(\text{n_samples}, |\text{memory}[s]|)$

2. **选择精英样本**：按奖励排序，取前 k 个最优动作：

$$\mathcal{E} = \text{top-}k\{a_j\}_{j=1}^{N} \text{ by } r_j, \quad k = \lfloor N \times \text{elite_frac} \rfloor$$

3. **计算精英统计量**：

$$\hat{\mu} = \frac{1}{k} \sum_{a \in \mathcal{E}} a, \quad \hat{\sigma} = \sqrt{\frac{1}{k} \sum_{a \in \mathcal{E}} (a - \hat{\mu})^2}$$

4. **平滑更新分布参数**（学习率 $\alpha = 0.3$）：

$$\mu_s \leftarrow (1 - \alpha)\mu_s + \alpha\hat{\mu}$$

$$\sigma_s \leftarrow \max\left(\sigma_{\min}, (1 - \alpha)\sigma_s + \alpha\hat{\sigma}\right)$$

5. **标准差衰减**（每个episode后）：

$$\sigma_s \leftarrow \max(\sigma_{\min}, \sigma_s \times \text{std_decay})$$

11.3 关键超参数

参数	默认值	说明
`n_samples`	100	每次评估时使用的最近经验数量
`elite_frac`	0.3	精英样本比例，取奖励最高的前30%
`initial_std`	20.0	初始标准差（价格探索范围约 ±40元）
`min_std`	2.0	最小标准差（收敛后最小探索范围）
`std_decay`	0.99	标准差衰减率，每个episode衰减1%
`memory_size`	100	每个状态的经验回放缓冲区大小
更新率 α	0.3	分布参数的平滑更新步长

11.4 探索与利用

CEM的探索机制天然内嵌于分布的标准差中：

- **训练初期**：σ_s 较大（20.0），采样范围广，充分探索价格空间
- **训练中期**：σ_s 通过衰减逐渐减小，探索范围收窄
- **训练后期**：σ_s 接近 $\sigma_{\min}$（2.0），策略趋于确定性

确定性评估时，直接返回分布均值 μ_s。

11.5 Agent的CEM配置差异

由于OTA的动作空间（补贴比例 [0, 0.8]）远小于酒店的价格空间（[80, 180]），OTA使用不同的CEM参数：

参数	酒店CEM	OTA CEM
`initial_std`	20.0	0.2
`min_std`	2.0	0.02
`action_min`	80.0	0.0
`action_max`	180.0	0.8

---

12. 关键设计决策总结

12.1 为什么选择CEM而非DQN/PPO等算法?

- **稳定性**：CEM不存在Q值高估、策略振荡等深度RL常见问题
- **无需梯度**：ABM环境是黑箱模拟器，奖励信号不可微分
- **适合随机环境**：CEM通过精英采样自然平滑了环境随机性

- **简洁高效**：表格版CEM实现简单，计算开销小
- **可解释性**：CEM的分布参数（均值和标准差）直接对应Agent在各状态下的定价策略和不确定性

12.2 为什么采用双CEM架构?

酒店的线上和线下价格面向不同的市场和约束条件：

- 线上价格需考虑佣金成本和OTA补贴行为
- 线下价格直接面向消费者，无中间商

两个独立的CEM算法可以让每个渠道的定价策略独立演化，避免单一CEM在高维联合空间中的效率问题。

12.3 决策桶的设计意义

91天逐日定价会产生 $91 \times 18 = 1638$ 个状态-动作对（不含阶段扩展），CEM需要大量样本才能收敛。通过将91天聚合为8个决策桶：

- 减少了状态空间（$8 \times 18 = 144$ 个状态-动作对）
- 符合酒店定价实践（近期精细调价，远期粗略定价）
- 保证了每个状态有足够的经验样本进行CEM更新

12.4 混合奖励的博弈论意义

混合奖励机制（α 参数）对应博弈论中的**社会福利函数**设计：

- $\alpha = 1$：纯纳什均衡方向，双方完全自利
- $\alpha = 0$：社会最优方向，最大化系统总利润
- $0 < \alpha < 1$：介于两者之间，可调节博弈的合作程度

通过调整 α 值，可以研究不同合作程度下的均衡策略变化。

> **文档版本**: v1.0
>
> **最后更新**: 2026年3月16日
>
> **适用代码版本**: 基于CEM算法的酒店-OTA博弈系统